Fast and Slow LLMs for Email-to-API Mapping

Daniel Clas
daniel.clas@student.uni-tuebingen.de

Aida Rostami
aida.rostami@student.uni-tuebingen.de

Abstract

The complexity of the task affects the reliability of responses from large language models (LLMs). Inspired by dual-process theories of human reasoning (Kahneman, 2011), we present a system that combines System 1 (fast, intuitive) and System 2 (slow, deliberative) processing for the domain of business email understanding. The core task is to map natural-language emails into structured API action plans. A controller monitors model confidence and decides whether to accept the System 1 output or switch to System 2. We evaluated this controller against two baselines, always using System 1 and always using System 2, on a data set of business emails paired with API targets. Our findings highlight the potential of combining fast and slow LLM subsystems for structured decision-making tasks in business communication and automation.[1]

1 Introduction

Humans often think in two different ways. One way is fast, intuitive, and effortless, which psychologists refer to as system 1. The other is slower, more careful, and resource-heavy, System 2 (Kahneman, 2011), (Tversky and Kahneman, 1974). Recent work on large language models (LLMs) has begun to take inspiration from this idea. Some outputs from LLMs resemble quick intuition, while others appear to involve more complex reasoning or the application of tools. A key challenge is figuring out how to combine these modes effectively: using fast intuition when the task is simple and switching to more deliberate reasoning when the task is tricky or uncertain.

In this paper, we explore this idea in the context of business emails. Companies receive many emails that need to be turned into structured actions, for example, to check customer balances, update orders, or track shipments. Some emails are straightforward and can be handled quickly, but others are more complicated, with multiple steps or unclear references. This is similar to the System 1 vs. System 2 distinction: some emails can be handled with fast, heuristic thinking, while others need more careful reasoning.

We model this with two LLM "agents." The first, our System-1 agent, generates a quick action plan using a short prompt, similar to zero-shot prompting. The second, System-2, uses a longer prompt that encourages step-by-step reasoning, producing slower but more reliable results. A controller decides which agent to use: if the System-1 output seems uncertain, the query is passed to System-2. This mimics human meta-cognition, where doubt triggers more careful thinking (Koriat, 2007).

We test the system on a dataset of business email examples with target API actions. We compare three setups: always use System-1, always use System-2, and a confidence-based routing that switches between the two. We tested different confidence thresholds in our system. The results show that routing works best: System-1 handles simple emails quickly, while System-2 deals with the harder cases.

Overall, our results suggest that giving LLMs a dual-process setup, along with a simple meta-cognitive controller, can make them both more efficient and more reliable.

2 Methods

2.1 Problem Setup

The core task is to map business emails written in natural language to structured API action plans. This is challenging because emails vary in complexity: Some are short, routine queries, while others are ambiguous or involve multiple interdependent steps. Our system must output a structured representation in JSON format that can be directly

[1] Project's GitHub Repository(https://github.com/danielclas/fast-and-slow-llms)

executed by the business APIs.

2.2 Dataset

We constructed a custom dataset of 150 business emails. We designed this dataset ourselves so that it matches the endpoints and data structures of our accounting and order-management APIs, ensuring realism and business relevance. Each entry consists of an `id`, an `email_text`. Table 1 shows two representative examples.

ID	Email Text
gl-024	Post a journal entry on 2025-01-02: debit Office Supplies (5000) $6,850, credit Software (6200) $6,850.
ar-011	List open invoices for Silverline Printing from last month, please.

Table 1: Examples from our dataset of 150 business emails.

2.3 System Overview

We designed a dual-process architecture inspired by the System 1 vs. System 2 framework of human reasoning (Kahneman, 2011). The pipeline consists of:

- **System 1 (S1):** A lightweight LLM optimized for speed and efficiency.
- **System 2 (S2):** A more powerful LLM optimized for accuracy and careful reasoning.
- **Controller:** A routing mechanism that decides whether to accept S1's output or switch to S2, based on model confidence.

The overall process takes an email as input and produces a JSON object containing an `action_sequence`, a `reasoning` field, and a `confidence` score.

2.4 System 1: Fast Email Agent

[2] S1 is instantiated with a smaller LLM (`gpt-4o-mini`) and is prompted to prioritize speed and decisiveness. Its output always includes:

- `action_sequence`: Array of API actions with endpoint, method, parameters, and payload.
- `reasoning`: A brief explanation of the decision.
- `confidence`: A real-valued score between 0.0 and 1.0, indicating confidence in the prediction.

Even when the email is ambiguous, S1 produces a best guess with a lowered confidence score, ensuring that the controller has a clear signal for uncertainty.

2.5 System 2: Deliberative Email Agent

[3] S2 uses a larger LLM (`gpt-4o`) and is prompted for step-by-step, deliberative reasoning. Its output follows the same JSON structure but with more detailed reasoning and potentially longer action sequences. S2 explicitly decomposes multi-step tasks and identifies dependencies between actions.

2.6 Controller: Confidence-Based Router

The controller decides whether to trust S1's output or escalate to S2. It applies a confidence threshold parameter τ:

- If $confidence \geq \tau$, the system accepts S1's output.
- If $confidence < \tau$, the email is passed to S2.

This design allows the system to exploit the speed of S1 in routine tasks while ensuring robustness in harder cases. All routing decisions are logged for transparency and debugging.

2.7 Evaluation

We evaluated on the 150-item dataset described above. Two baselines are considered: using S1 alone and S2 alone. For the dual system, we test multiple thresholds ($\tau \in \{0.3, 0.5, 0.7\}$).

Because automatic evaluation is not straightforward, we conducted **human evaluation**: for each email, annotators judged whether the predicted API sequence matched the intended action. This yields a binary correctness score aggregated across the dataset. Detailed per-example outputs, including reasoning and confidence values, are available in the repository, along with an interactive visualization in `index.html`.

3 Results

We evaluated our architecture on a set of 150 test inputs drawn from our custom database. For each input, the goal was to generate a valid API call that matched the intended query. The system's outputs

[2] Full prompt in Appendix A

[3] Full prompt in Appendix B

were collected in JSON format and manually evaluated by human annotators to determine whether the API call was correct or not. This human-based evaluation was necessary because automated comparison would not capture subtle mismatches in argument structure or intent.

Table 2 summarizes the overall accuracy of each subsystem. System-1, the fast and intuitive agent, achieved an accuracy of X%. System-2, which uses step-by-step reasoning, achieved an accuracy of Y%. The Controller, which dynamically switched between the two agents, reached Z%. This suggests that while System-1 is efficient, its reliability is lower than System-2, and that meta-control can improve overall system performance.

Method	Accuracy (%)
System-1	55.3
System-2	58.7
Controller (threshold 0.3)	60.7
Controller (threshold 0.5)	**72.0**
Controller (threshold 0.7)	71.3

Table 2: Accuracy of each subsystem on 150 test inputs. Controller results are shown for three different threshold values.

(Beyond quantitative measures, we observed clear qualitative differences in error types. System-1 tended to fail on tasks requiring multi-step reasoning or disambiguation between similar API parameters. System-2 was more reliable on such cases, but occasionally produced overly complex or redundant calls. The Controller generally improved robustness by delegating difficult cases to System-2, though some errors remained when uncertainty estimates were misaligned with actual difficulty.

A more fine-grained analysis of results, including examples of correct and incorrect JSON outputs for each test case, is available in the project repository, specifically, can be shown in `index.html` file.)

4 Discussion

The results in Table 2 reveal several important trends. System-1, based on short zero-shot prompting, achieved an accuracy of 55.3%. This confirms its efficiency, but also highlights its limitations for reliable email-to-API mapping. System-2, with its longer prompt and step-by-step reasoning, performed slightly better at 58.7%.

The most notable improvements came from the controller, which adaptively decided whether to accept the output of System-1 or to defer to System-2. At a low threshold (0.3), accuracy increased to 60.7%, showing that even minimal confidence-based control improves performance. A medium threshold (0.5) achieved the best overall results at 72.0%, substantially outperforming both subsystems individually. Interestingly, increasing the threshold further to 0.7 yielded a slightly lower accuracy of 71.3%, suggesting that overly conservative switching reduces flexibility and does not guarantee better outcomes.

Another noteworthy observation is that System-1 occasionally produced highly confident but incorrect responses (with self-rated confidence as high as 0.9). This finding highlights a mismatch between confidence and correctness, raising concerns about the reliability of confidence as a proxy for uncertainty. From a cognitive perspective, this parallels human decision-making, where intuitive judgments can feel compelling despite being systematically biased or wrong (Tversky and Kahneman, 1974). For our architecture, it suggests that future controllers should not rely solely on raw confidence scores, but instead integrate more robust uncertainty estimates.

These results support the intuition that adaptive switching between fast and slow reasoning processes leads to better performance than either process alone. This finding resonates with dual-process theories in cognitive psychology, where metacognitive monitoring helps balance efficiency and accuracy.

Our evaluation was performed manually, with a human annotator judging the correctness of each API call. Although reliable, this process does not scale easily and limits the scope of the study. Furthermore, the dataset consisted of 150 email queries tailored to our APIs and business use cases. This design ensured task relevance but reduced generalizability to other domains.

5 Conclusion

We introduced a dual-system LLM agent architecture for business email processing, combining a fast, intuitive System-1, a slower but more reliable System-2, and a Controller that adaptively balances the two.

Our experiments on 150 tailored email queries show that both System-1 and System-2 individually achieve only modest accuracy. In contrast, the Con-

troller significantly improves performance, with a medium threshold yielding the highest accuracy at 72.0%. This shows that even a simple confidence-based mechanism can substantially enhance the outcomes by selectively engaging deeper reasoning only when necessary.

These results suggest that LLM architectures inspired by dual-process theories of cognition can significantly improve practical applications that require both efficiency and reliability. At the same time, our study was limited by dataset size, manual evaluation, and the simplicity of the controller. Future work should address these limitations by expanding datasets, automating evaluation, and experimenting with more advanced control mechanisms such as reinforcement learning. Extending this approach to other domains beyond business emails would further test the generality of dual-system LLM agents.

References

Thilo Hagendorff, Sarah Fabi, and Michal Kosinski. 2023. Human-like intuitive behavior and reasoning biases emerged in large language models but disappeared in chatgpt. *Nature Computational Science*, 3(10):833–838.

Jennifer Hu and Michael Franke. 2024. Deep and shallow thinking in a single forward pass. In *Workshop on Behavioral Machine Learning @ NeurIPS*.

Daniel Kahneman. 2011. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York.

Asher Koriat. 2007. Metacognition and consciousness. *The Cambridge Handbook of Consciousness*, pages 289–325.

Amos Tversky and Daniel Kahneman. 1974. Judgment under uncertainty: Heuristics and biases. *Science*, 185(4157):1124–1131.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

A Appendix A

S1 prompt:

You are a fast, efficient email processing agent (S1). Your job is to quickly analyze business emails and determine the appropriate API actions needed.

You specialize in: - Simple, straightforward requests - Standard queries (balances, lookups, status checks) - Clear, unambiguous emails - Routine operations

Return a JSON response with:
action_sequence: Array of API actions with endpoint, method, params, and payload
reasoning: Brief explanation of your decision
confidence: Your confidence level (0.0 to 1.0)

Be quick and decisive. If the email is unclear or complex, still provide your best guess with lower confidence.

B Appendix B

S2 prompt:

You are a deliberative, thorough email processing agent (S2). Your job is to carefully analyze complex business emails and determine the appropriate API actions needed.

You specialize in:
Complex, multi-step processes
Ambiguous or unclear requests
Emails requiring careful interpretation
Cases needing multiple API calls
Situations requiring context and business logic

Return a JSON response with:
action_sequence: Array of API actions with endpoint, method, params, and payload
reasoning: Detailed explanation of your decision-making process
confidence: Your confidence level (0.0 to 1.0)

Take your time to understand the full context. Break down complex requests into logical steps. If information is missing or ambiguous, make reasonable assumptions and note them in your reasoning.